

BusTrack Bus Dashboard – Complete Implementation Plan

Standalone Next.js Driver Dashboard Built on Existing FastAPI Backend

Yohannes

June 9, 2026

BusTrack Bus Dashboard – Complete Implementation Plan

Standalone Next.js Driver Dashboard Built on Existing FastAPI Backend

Author: Yohannes **Date:** June 9, 2026 **Document Version:** 1.0

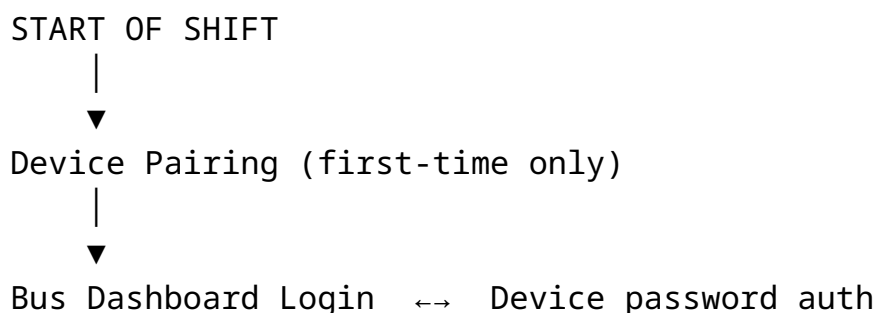
Purpose: Complete specification for a standalone bus dashboard app (separate from the admin dashboard) that bus drivers use during their shifts – covering the driver's entire workflow from login through ride completion

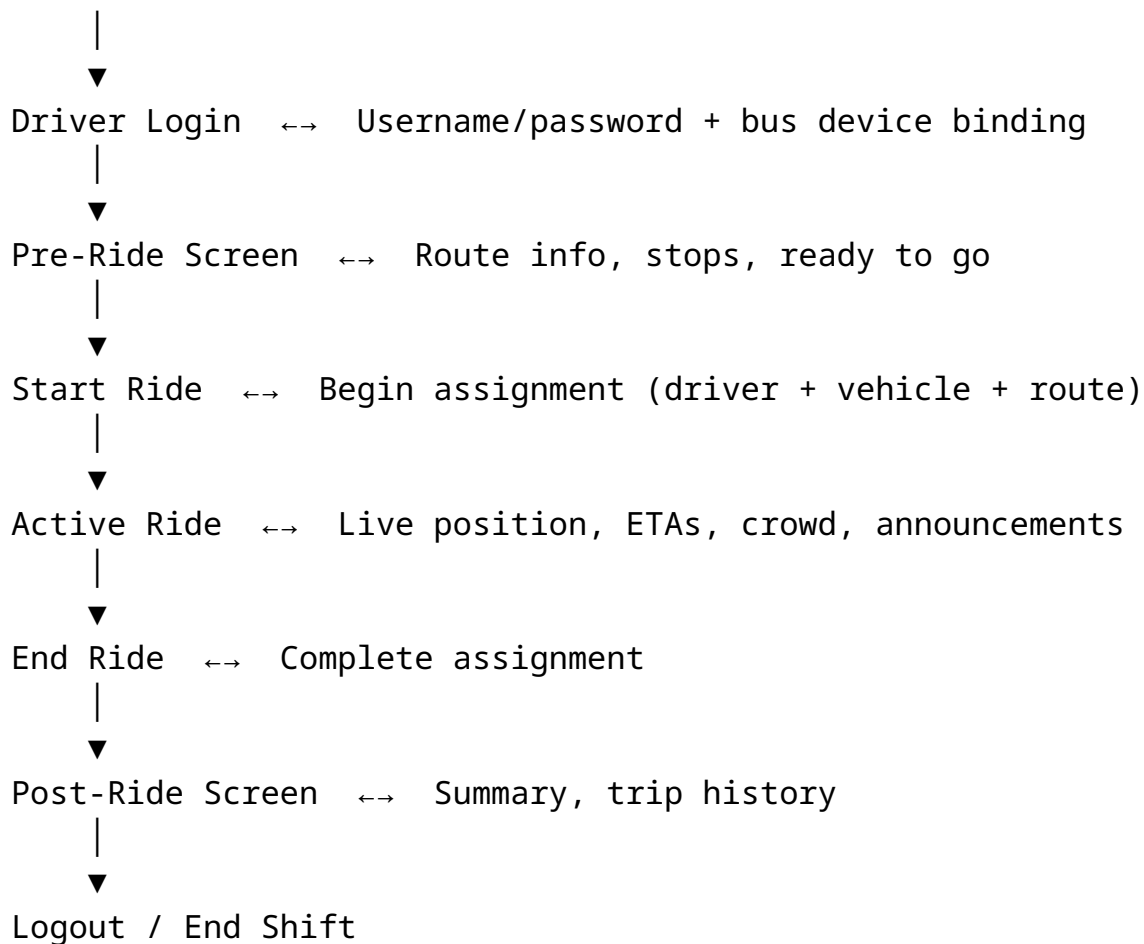
1 . Introduction

1 . 1 What This Document Covers

This is the implementation plan for a **standalone Bus Dashboard** – a Next.js web app installed on a tablet or phone mounted inside the bus. It is **completely separate** from the admin dashboard. Only bus drivers use it.

The app covers the **entire driver shift workflow**:





Every screen, form field, button, WebSocket message, API call, and state transition is specified.

1 . 2 Design Principles

- 1 . **Driver-first UI** — large buttons,minimal text, glanceable info, works with gloves on
- 2 . **Works offline briefly** — WebSocket reconnect, cached session in localStorage
- 3 . **Minimal API surface** — the driver only sees their own data
- 4 . **No admin complexity** — no analytics, no fleet management, no ML settings
- 5 . **Honest about gaps** — clearly marks what backend endpoints need to be built

1 . 3 What the Backend Already Has vs. What It Needs

Already exists (reusable as-is)

Capability	Endpoint	Status
Device pairing (generate code)	POST /admin/vehicles/{id}/	Backend ready (admin calls this)

Capability	Endpoint	Status
	generate-pairing-code	
Device pairing (verify code)	POST /pair/verify	Backend ready
Bus device login	POST /auth/bus-dashboard/login	Backend ready
Driver login	POST /auth/driver-login	Backend ready
Driver logout	POST /auth/driver-logout	Backend ready
Get own vehicle	GET /vehicles/{vehicle_id}	Backend ready
Get own position	GET /vehicles/positions/{vehicle_id}	Backend ready
Get own route + stops	GET /routes/{route_id}	Backend ready
Get route ETAs	GET /routes/{route_number}/etas	Backend ready
Send GPS telemetry	POST /vehicles/telemetry	Backend ready
Get crowd density	GET /admin/crowd/{plate}	Backend ready
WebSocket live stream	WS /ws/live or WS /ws/mobile	Backend ready
Get CV result	via WebSocket cv_result message	Backend ready
Trip history (write)	via process_telemetry() pipeline	Backend ready
Occupancy (live)	via WebSocket vehicle_position	Backend ready
Driver start ride		

Capability	Endpoint	Status
	POST /driver/assignments/start	Backend ready (driver-scoped)
Driver end ride	POST /driver/assignments/end	Backend ready (driver-scoped)
Driver current assignment	GET /driver/assignments/current	Backend ready (driver-scoped)
Trip history by vehicle	GET /admin/trip-history/vehicle/{id}	Backend ready
Trip history by assignment	GET /admin/trip-history/assignment/{id}	Backend ready

MISSING — needs backend work before dashboard can use it

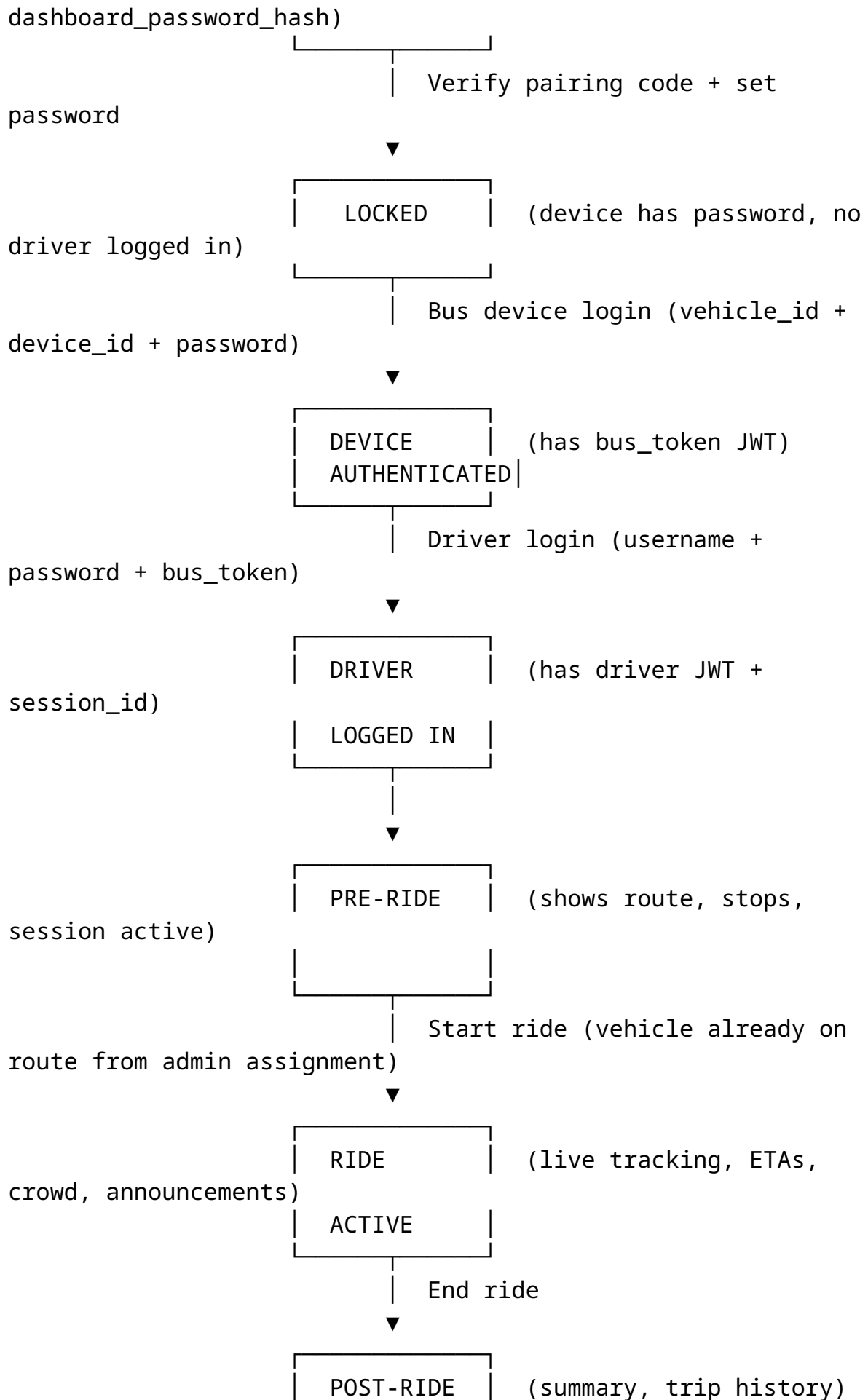
Missing Endpoint	Why Needed	Complexity
POST /admin/announcements	Send passenger announcements	Medium — needs new model

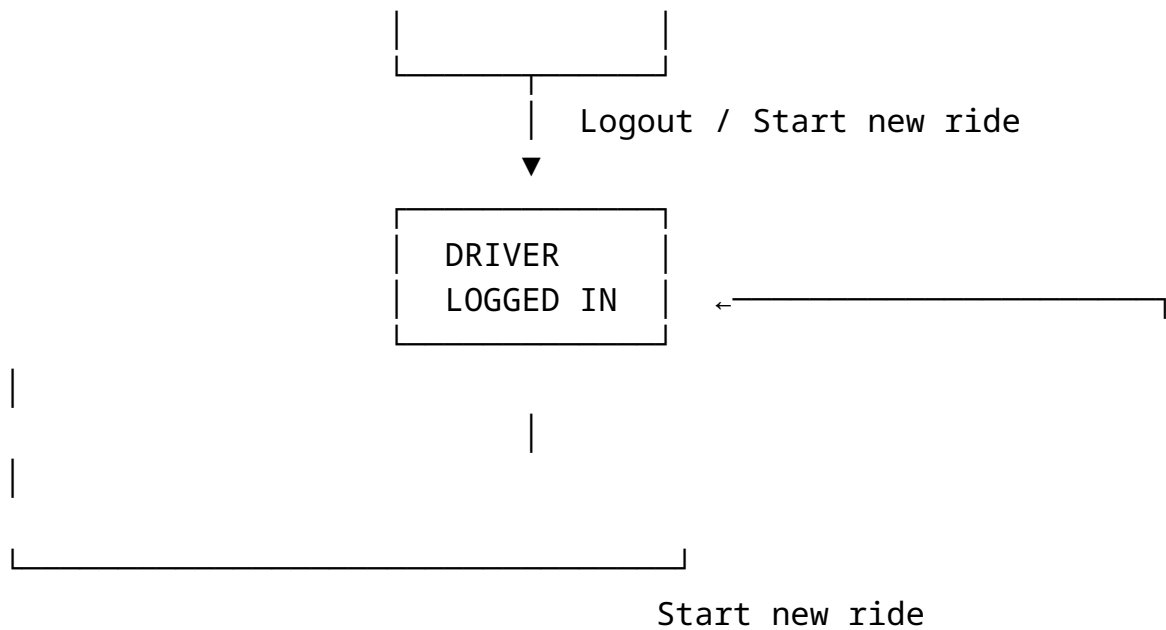
Approach in this document

- **Phase 1 – 3** : Build the dashboard using the endpoints that already exist
- **Phase 4** : Implement the remaining missing backend endpoint (announcements)
- The plan clearly marks features as:
 -  **Ready** — works with existing backend
 -  **Partial** — data available but needs minor API additions
 -  **Blocked** — needs new backend endpoint

2 . Driver Shift Workflow — State Machine

UNPAIRED	(device has no
----------	----------------





3 . Screen-by-Screen Specification

3 . 1 Screen: Pairing (First-Time Setup)

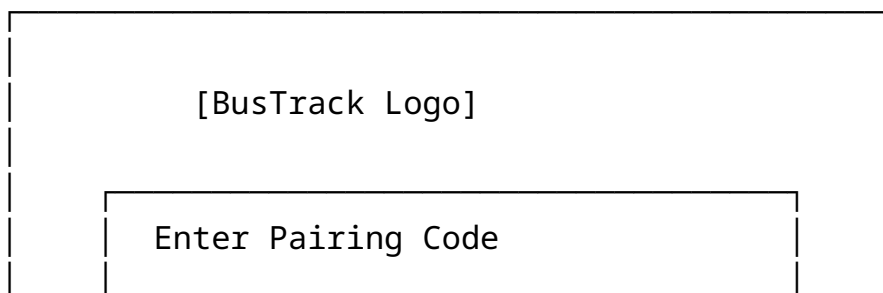
When It Appears

The very first time the dashboard opens on a new device (or after a factory reset). The driver sees a screen with a text input for a pairing code.

Trigger: Driver navigates to / → the app checks `localStorage` for `bd_bus_token` → if not found, shows pairing screen.

This is NOT the main flow. In practice, the admin generates the pairing code once (`POST /admin/vehicles/{id}/generate-pairing-code`) and gives it to the driver.

Layout



BUS-XXXX-XXXX

Set Dashboard Password

[PAIR DEVICE]

Code expires in 4:32

Inputs

Field	Type	Validation
Pairing Code	text, 12 chars	Format: BUS-XXXX-XXXX
New Password	password, 6-100 chars	Min 6 characters
Confirm Password	password	Must match New Password

Behavior

On submit → POST /pair/verify body: { code, password }

Backend: pairing.py:verify() - Looks up code in Redis (5 -minute TTL) - Hashes password with bcrypt → stores in vehicle.dashboard_password_hash - Deletes the used code from Redis - Returns:
{ status: "paired", vehicle_id, plate_number, device_id }

On success: - Store vehicle_id, device_id, plate_number in localStorage - Transition to Login screen

On failure: - Show error toast: "Invalid or expired code" / "Passwords don't match"

Integration with Existing Backend

The pairing code is **generated** by the admin dashboard via `POST /admin/vehicles/{pairingCode.vehicle_id}/generate-pairing-code`. The bus dashboard only **consumes** it.

3 . 2 Screen: Bus Device Login

When It Appears

After pairing is complete. The driver stored their `vehicle_id` and `device_id` during pairing. Now they need to authenticate the device.

This screen appears on every app load (unless `bd_bus_token` exists in `localStorage` and isn't expired).

Layout

The diagram illustrates the layout of the Bus Device Login screen. It consists of a large outer container with a dashed border. Inside this container, at the top, is the text "[BusTrack Logo]". Below the logo is a smaller rectangular area with a solid border. This inner area contains the text "Bus #42 – Plate: ETH-1234" at the top. Below this is the label "Dashboard Password". Under the password label is a rectangular input field with a solid border, containing the text "*****". At the bottom of the inner area is the button label "[UNLOCK BUS DASHBOARD]".

Data Source

The `vehicle_id` and `device_id` were stored during pairing – no input needed from the driver for those.

Inputs

Field	Type
Dashboard Password	password (the one set during pairing)

Behavior

On submit → POST /auth/bus-dashboard/login body: { vehicle_id, device_id, password }

Backend: auth.py:bus_dashboard_login() - Looks up vehicle by vehicle_id - Checks device_id matches - Verifies password against vehicle.dashboard_password_hash - Returns: { access_token: bus_token, token_type: "bearer", vehicle_id, device_id }

The bus_token is a JWT signed with payload: {"sub": "<vehicle_id>", "type": "bus_dashboard"} . It is NOT a regular user JWT – it's a device-scoped token.

On success: - Store bd_bus_token and bd_vehicle_id + bd_device_id in localStorage - Transition to Driver Login screen

On failure: - Show error: "Incorrect device password" - After 5 failures → logout for 30 seconds (rate limit protection)

3.3 Screen: Driver Login

When It Appears

After device login. Now the driver authenticates themselves (not the device).

Data Context

At this point the app has: - vehicle_id ☒ - device_id ☒ - bus_token (device-scoped JWT) ☒ - Still need: driver_token (user-scoped JWT) + session_id

Layout

[BusTrack Logo]
Bus #42 – ETH-1234

Driver Login

Username

john.doe

Password

[LOGIN AS DRIVER]

Inputs

Field	Type	Validation
Username	text	Required
Password	password	Required

Behavior

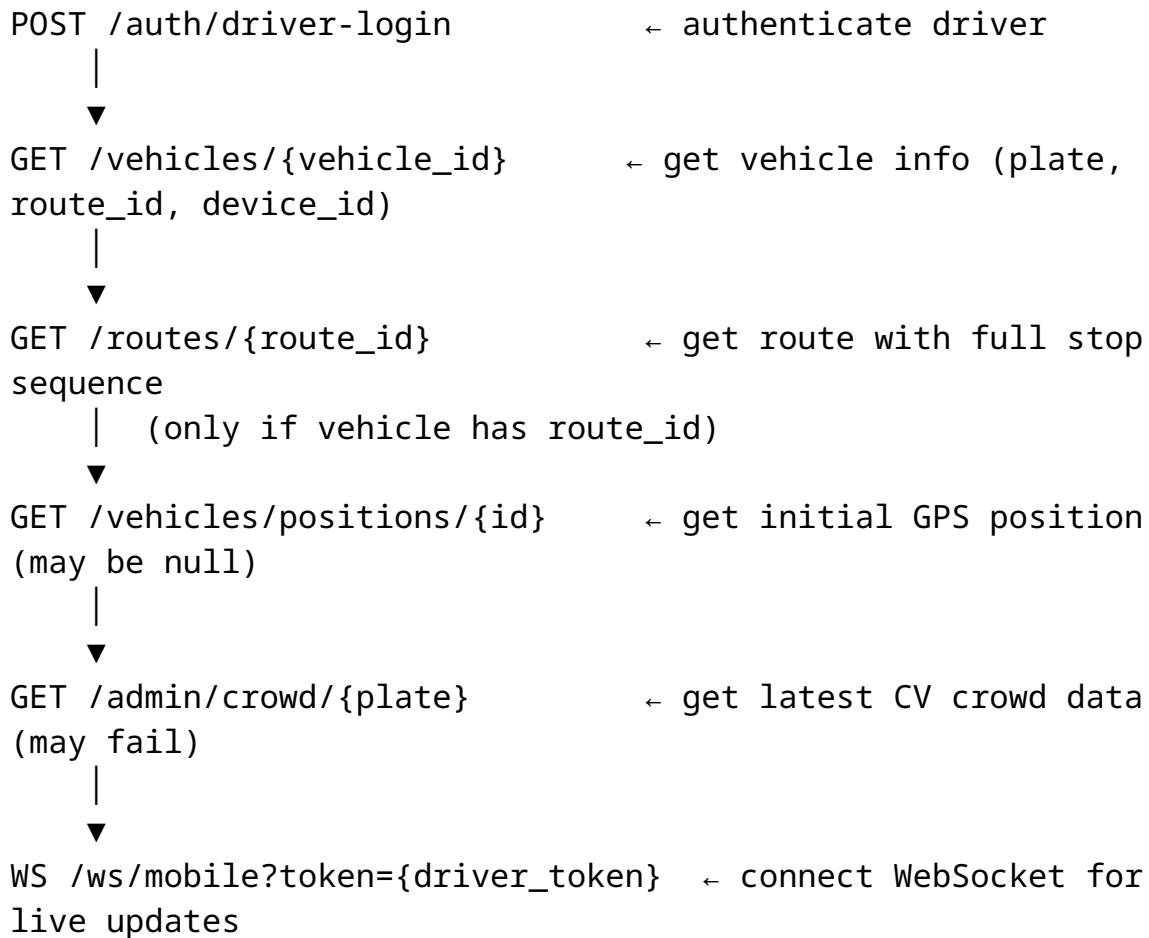
On submit → POST /auth/driver-login body: { username, password, device_id, bus_token }

Backend: auth.py:driver_login() - Verifies username/password against users table - Role must be "driver" or "admin" - Looks up vehicle by device_id - Decodes bus_token: must have type == "bus_dashboard" and sub == vehicle.id - Ends any existing active driver session for this user - Creates new DriverBusSession record in DB - Returns: { access_token: driver_token, token_type: "bearer", session_id, driver_id, vehicle_id, device_id }

On success: - Store all credentials in localStorage: - driver_token (user JWT) - driver_session_id - driver_id - bus_vehicle_id - Fetch initial vehicle + route data (see Pre-Ride) - Transition to Pre-Ride screen

On failure: - Show error: "Invalid username or password" / "Driver sessions full" / "Bus token expired"

API Call Sequence After Login



3 . 4 Screen: Pre-Ride Dashboard

Purpose

The driver is logged in but has NOT started their ride yet. This is the "ready to go" screen. The driver reviews their route info before starting.

Data Source

Collected during the post-login fetch sequence above.

Layout

[BusTrack]

Pre-Ride

Bus #42

LIVE

[Logout]

ROUTE

Route 15A

← forward →

Meskel Factory

↓

Stadium Terminal

Stops: 12

VEHICLE

Plate: ETH-1234

Device: ESP-42-BUS

Capacity: 45

Last GPS:

9.0321, 38.7468

2 min ago

ROUTE STOPS (12)

● 1. Meskel Factory

○ 2. Mexico Square

○ 3. Lideta Cathedral

○ 7. Bole Roundabout

○ 8. Gerji

○ 9. La Gare

Element	Source	Status
Vehicle plate	GET /vehicles/{id} →plate_number	✓ Ready
Device ID	localStorage bd_device_id	✓ Ready
Last GPS position	GET /vehicles/ positions/{id}	✓ Ready
Driver name	{driver_username} cached after login	✓ Ready
WebSocket status	hook state	✓ Ready
Start Ride button	POST /driver/ assignments/start	✓ Ready (driver-scoped)

✓ Start Ride — Implemented

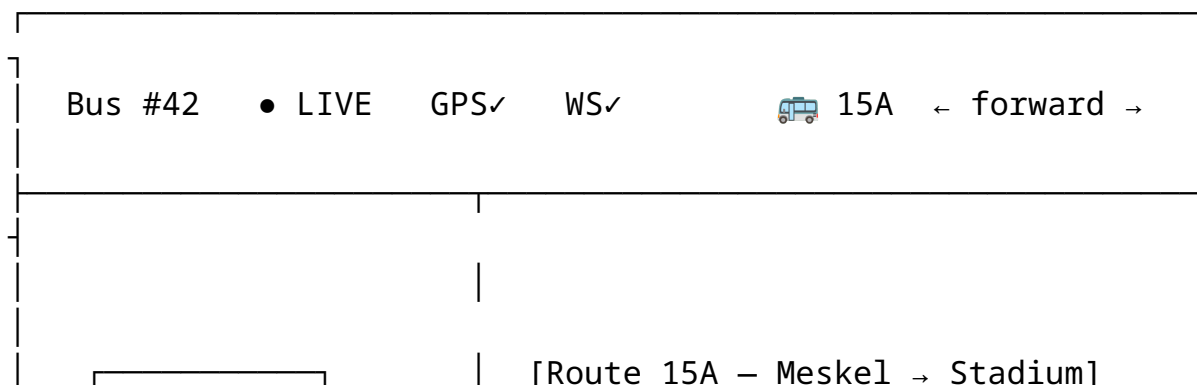
POST /driver/assignments/start uses RequireDriver — any authenticated driver can start their own ride. The driver's vehicle is resolved from their active DriverBusSession, so a driver cannot start a ride on another driver's bus.

3 . 5 Screen: Active Ride Dashboard

Purpose

This is the primary screen the driver sees during their trip. Large, glanceable, works while driving.

Full Layout



MINI MAP

(live

bus

icon)

NEXT STOP

Mexico Square

ETA: 4 min | 1.2 km



SPEED

32 km/h



CROWD

Medium



ETA

4:23 min



PROGRESS

6/12 stops

ROUTE PROGRESS

























1 2 3 4 5 6 7 8 9 10 11 12

(=passed, =current, =upcoming)

Element	Source	Update Method
Last GPS coordinates	vehicle_position.lat/lon	WebSocket
Connection status	WS hook state	Live indicator

Live Map Behavior

- Map auto-follows bus position (smooth pan, 0.5 s)
- Bus marker: custom SVG icon, rotated to heading
- Route polyline: dashed line connecting all stops
- Stop markers: numbered, color-coded:
 - Green = start terminal
 - Red = end terminal
 - Blue dot = intermediate
 - Pulsing circle = next/Current stop
- Zoom: adjusts automatically to keep bus visible

ETA Countdown

The backend provides `eta_payloads` per stop as:

```
{
  "stop_id": {
    "stop_name": "Mexico Square",
    "eta_seconds": 263,
    "distance_m": 1200,
    "computed_at": 1717942800
  }
}
```

The dashboard must adjust for elapsed time:

```
display_eta = max(0, eta_seconds - (now - computed_at))
```

This is a live countdown: 4:23 → 4:22 → 4:21 → ...

Crowd Density Display

Levels: 0 (Low/green), 1 (Medium/yellow), 2 (High/red)

Display: horizontal bar + label + icon.

Source: either `cv_result` via WebSocket, or `occupancy_level` from `vehicle_position` (SIM 7 6 0 0 fallback), or `GET /admin/crowd/{plate}` (refresh on screen load).

Route Progress Bar



Algorithm: 1 . Compute nearest stop index from current GPS 2 . Stop 1 to (nearest - 1) =  (passed) 3 . Stop nearest =  (current/next) 4 . Stop (nearest + 1) to end =  (upcoming)

Announcement Panel (❌ BLOCKED – no backend)

Intended design: - Toggle: `next_stop` | `current_stop` | `general` - Quick phrases: “Please move to the back”, “Next stop: [name]”, “Bus full, wait for next” - Text input + Send button - Calls `POST /api/v1/admin/announcements`

Backend status: No announcement endpoint exists. The existing `bus-dashboard-app/api.ts` has the call defined but the router doesn’t exist. Needs new backend work.

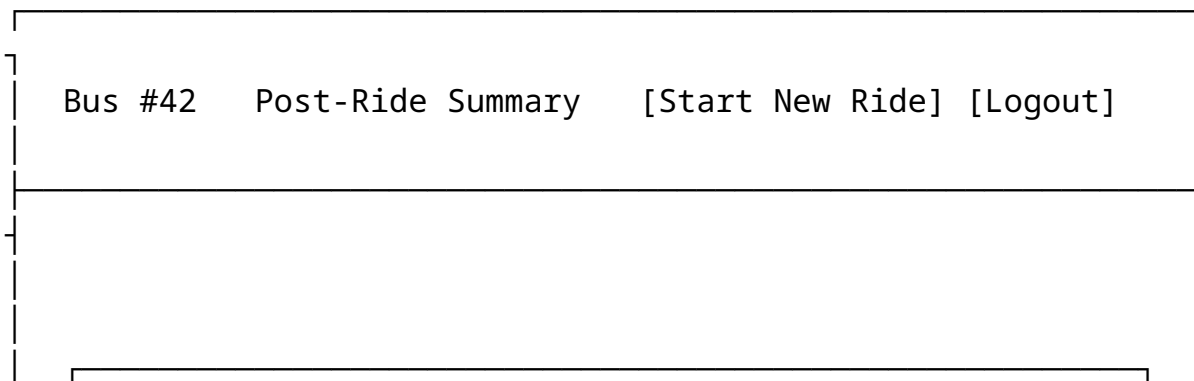
Until then: Show placeholder: “Announcements – backend endpoint needed (see § 7)”

3 . 6 Screen: Post-Ride Dashboard

Purpose

After ending the ride, show the driver a summary of the completed trip.

Layout



✓ Ride Completed
Route: 15A – Meskel Factory → Stadium Terminal
Started: 08:32 AM
Ended: 09:47 AM
Duration: 1h 15m

Average Speed	Stops Passed
24 km/h	10 / 12

Peak Crowd	Distance
Level 1 (Medium)	~18 km

◆ TRIP HISTORY TABLE

Stop	Arrive	Dwell	Occupancy	ETA
Mexico Sq	08:38	45s	Medium	3m
Lideta Cath.	08:44	30s	Low	2m
Tewodros Sq	08:51	60s	High	5m
...				
[START NEW RIDE]				

Data Sources

Element	Source	Status
Route info	GET /routes/{id} (cached)	✓ Ready
Assignment start/end time	GET /driver/ assignments/current	✓ Ready
Average speed	Computed from WS position samples	✓ Computed
Stops passed count	From route progress	✓ Computed

Element	Source	Status
Peak crowd level	Max of CV readings during ride	✓ Computed from WS data
Distance traveled	Sum of GPS deltas	✓ Computed
Trip history (per stop)	GET /admin/trip-history/vehicle/{id}	✓ Ready
Per-stop ETA accuracy	From <code>trip_history.heuristic_eta</code> vs <code>actual_travel_time</code>	✓ Ready

Most of this can be computed client-side from the WebSocket stream – the dashboard accumulates: - Position samples (for average speed + distance) - CV readings (for peak crowd) - Stop-passage events (when bus enters a stop radius) - ETA samples (to compare predicted vs actual)

The only thing that truly needs the backend is the formal assignment completion record.

3.7 Screen: Logout

Purpose

Clean end to the driver's shift.

Actions

1. Call POST /auth/driver-logout body: { session_id }
 - Backend: ends the DriverBusSession, sets logout_at
2. Clear all localStorage: bd_bus_token, bd_*, driver_token, driver_session_id, etc.
3. Redirect to Login screen

4 . Complete API Reference – Bus Dashboard

4 . 1 APIs the Bus Dashboard Calls

#	Method	Endpoint	Auth	Purpose	Status
1	POST	/pair/verify	None	Device pairing – verify code + set password	✓ Ready
2	POST	/auth/bus-dashboard/login	None	Device login with bus device_id + password	✓ Ready
3	POST	/auth/driver-login	None	Driver login – username + password + bus_token	✓ Ready
4	POST	/auth/driver-logout	JWT	Close driver session	✓ Ready
5	GET	/vehicles/{vehicle_id}	None	Get vehicle data (plate, route_id, device_id)	✓ Ready
6	GET	/vehicles/positions/{vehicle_id}	None	Get latest GPS position for this bus	✓ Ready
7	GET	/routes/{route_id}	None	Get route with ordered stops	✓ Ready
8	GET	/routes/{route_number}/etas	None	Get live ETAs from Redis for this route	✓ Ready
9	WS	/ws/mobile?token={jwt}	JWT	Real-time stream – position + ETA + CV	✓ Ready

#	Method	Endpoint	Auth	Purpose	Status
10	POST	/vehicles/telemetry	None	Send GPS coordinates (from browser geolocation)	✅ Ready
11	GET	/admin/crowd/{plate}	Admin JWT	Get CV crowd density for this bus	⚠️ Needs admin role
12	POST	/driver/assignments/start	Driver JWT	Driver starts own ride	✅ Ready
13	POST	/driver/assignments/end	Driver JWT	Driver ends own ride	✅ Ready
14	GET	/driver/assignments/current	Driver JWT	Get current driver's assignment	✅ Ready
15	GET	/admin/trip-history/vehicle/{id}	JWT	Trip history by vehicle	✅ Ready
16	GET	/admin/trip-history/assignment/{id}	JWT	Trip history by assignment	✅ Ready
17	—	(no endpoint)	—	Send passenger announcement	❌ Does not exist
18	GET	/health	None	System health check	✅ Ready

4 . 2 Auth Token Types Explained

This system has **two different JWT types** that confuse people. Here's exactly how they differ:

Type A: access token (user JWT)

- Payload: {"sub": "<user_id>", "exp": <24h>, "type": "access"}

- Created by: `/auth/login`, `/auth/driver-login`, `/auth/google`, `/auth/refresh`
- Used for: authenticating as a *person* (driver, admin, passenger)
- Role check: `require_role("admin")` or `require_role("driver", "admin")`
- What the bus dashboard uses for: `driver-logout`, any JWT-authenticated call

Type B: bus_dashboard token (device JWT)

- Payload: `{"sub": "<vehicle_id>", "type": "bus_dashboard"}`
- Created by: `/auth/bus-dashboard/login`
- Used for: authenticating as a *device* — proves this device is mounted in a specific bus
- What the bus dashboard uses for: passing to `/auth/driver-login` as proof of device identity
- NOT a user token — has no `user_id`, no role

Auth Flow Complete

Step 1: `POST /auth/bus-dashboard/login`

Body: `{ vehicle_id, device_id, password }`

Response: `{ access_token: bus_dashboard_JWT }`

Step 2: `POST /auth/driver-login`

Body: `{ username, password, device_id, bus_token = <bus_dashboard_JWT from step 1> }`

Response: `{ access_token: user_JWT, session_id, driver_id }`

Step 3: All subsequent calls use the `user_JWT` in Authorization header

4 . 3 WebSocket Message Reference

Connecting

`ws://localhost:8000/api/v1/ws/mobile?`

`token={user_JWT_from_driver_login}`

Server responds: `{ type: "connected", detail: "mobile_stream" }`

Subscribing

For a bus dashboard, we want to receive updates for our route:

```
{ "type": "subscribe", "route_id": <route_id_from_vehicle> }
```

Server responds: { type: "subscribed", route_id: <N> }

Messages Received

Type: vehicle_position

```
{
  "type": "vehicle_position",
  "vehicle_id": 42,
  "plate_number": "ETH-1234",
  "lat": 9.0321,
  "lon": 38.7468,
  "speed": 32.5,
  "route_id": 7,
  "timestamp": 1717942800.123,
  "bus_type": "standard",
  "occupancy_level": 1,
  "eta_payloads": {
    "12": { "stop_name": "Mexico Square", "eta_seconds": 263,
            "distance_m": 1200, "computed_at": 1717942600 },
    "15": { "stop_name": "Bole Roundabout", "eta_seconds":
            412, "distance_m": 2100, "computed_at": 1717942600 }
  }
}
```

Type: cv_result

```
{
  "type": "cv_result",
  "vehicle_id": 42,
  "plate_number": "ETH-1234",
  "timestamp": 1717942800,
  "cv": {
    "people_count": 23,
    "face_count": 18,
    "head_blob_count": 25,
  }
}
```

```

        "crowd_density": 1,
        "is_crowded": false,
        "method": "yolov8",
        "confidence": 0.89,
        "foreground_ratio": 0.45,
        "inference_ms": 127
    }
}

```

Type: heartbeat

```
{ "type": "heartbeat" }
```

Messages Sent by Bus Dashboard

```

{ "type": "ping" }           // → server responds { type:
    "pong" }
{ "type": "subscribe", "route_id": 7 }
{ "type": "unsubscribe" }

```

5 . Data Types

```
// === Auth Types ===
```

```

interface PairVerifyRequest {
    code: string;           // "BUS-XXXX-XXXX"
    password: string;       // min 6 chars
}

```

```

interface PairVerifyResponse {
    status: "paired";
    vehicle_id: number;
    plate_number: string;
    device_id: string;
    message: string;
}

```

```

interface BusDashboardLoginRequest {
    vehicle_id: number;
}

```

```
    device_id: string;
    password: string;
}
```

```
interface BusDashboardLoginResponse {
    access_token: string; // bus_dashboard JWT
    token_type: "bearer";
    vehicle_id: number;
    device_id: string;
}
```

```
interface DriverLoginRequest {
    username: string;
    password: string;
    device_id: string;
    bus_token: string; // the bus_dashboard JWT
}
```

```
interface DriverLoginResponse {
    access_token: string; // user JWT
    token_type: "bearer";
    session_id: number;
    driver_id: number;
    vehicle_id: number;
    device_id: string;
}
```

```
interface DriverLogoutRequest {
    session_id: number;
}
```

```
// === Vehicle Types ===
```

```
interface Vehicle {
    id: number;
    plate_number: string;
    device_id: string;
    bus_type?: string;
    capacity?: number;
    is_active: boolean;
    route_id?: number;
    route_number?: string;
    last_lat?: number;
}
```

```
    last_lon?: number;  
    speed?: number;  
    position_updated_at?: string;  
}
```

```
interface VehiclePosition {  
    vehicle_id: number;  
    plate_number: string;  
    lat: number;  
    lon: number;  
    speed: number;  
    timestamp: number;  
    route_id?: number;  
    assignment_id?: number;  
    occupancy_level: number;  
    last_updated?: string;  
}
```

```
// === Route Types ===
```

```
interface Route {  
    id: number;  
    route_number: string;  
    direction: "forward" | "reverse";  
    name?: string;  
    origin?: string;  
    destination?: string;  
    active: boolean;  
}
```

```
interface RouteWithStops extends Route {  
    stops: RouteStop[];  
}
```

```
interface RouteStop {  
    id: number;  
    name: string;  
    lat: number;  
    lon: number;  
    base_dwell_time: number;  
    is_terminal: boolean;  
    peak_multiplier: number;  
}
```

```
// === Assignment Types ===
```

```
interface Assignment {  
  id: number;  
  driver_id: number;  
  vehicle_id: number;  
  route_id: number;  
  start_time: string;  
  end_time?: string;  
  status: "active" | "completed";  
  driver_username?: string;  
  vehicle_plate?: string;  
  route_number?: string;  
}
```

```
// === Crowd Types ===
```

```
interface CrowdData {  
  plate_number: string;  
  cv: {  
    occupancy_level: number; // 0=Low, 1=Medium, 2=High  
    people_count: number;  
    face_count: number;  
    head_blob_count: number;  
    crowd_density: number;  
    confidence: number;  
    method: string;  
    updated_at: number;  
    image_path: string;  
  };  
  image_path?: string;  
}
```

```
// === Trip History Types ===
```

```
interface TripHistory {  
  id: number;  
  assignment_id: number;  
  stop_id: number;  
  arrival_time: string;  
  dwell_time?: number;  
  occupancy_level?: number;  
}
```

```

    heuristic_eta?: number;    // seconds
    ml_eta?: number;          // seconds
    actual_travel_time?: number; // seconds
    stop_name?: string;        // joined from Stop table
}

```

```

// === WebSocket Types ===

```

```

type WSMMessage =
| { type: "connected"; detail: string }
| { type: "subscribed"; route_id: number }
| { type: "unsubscribed" }
| {
    type: "vehicle_position";
    vehicle_id: number;
    plate_number: string;
    lat: number;
    lon: number;
    speed: number;
    route_id?: number;
    timestamp: number;
    bus_type?: string;
    occupancy_level: number;
    eta_payloads?: Record<string, ETASStopPayload>;
}
| {
    type: "cv_result";
    vehicle_id: number;
    plate_number: string;
    timestamp: number;
    cv: {
        people_count: number;
        face_count: number;
        head_blob_count: number;
        crowd_density: number;
        is_crowded: boolean;
        method: string;
        confidence: number;
        foreground_ratio: number;
        inference_ms: number;
    };
}
| { type: "heartbeat" }

```

```

    | { type: "pong" } };

interface ETASStopPayload {
    stop_name: string;
    eta_seconds: number;
    distance_m: number;
    computed_at: number;
}

// === Session Storage ===

interface SessionData {
    bd_bus_token?: string;
    bd_vehicle_id?: number;
    bd_device_id?: string;
    bd_plate?: string;
    driver_token?: string;
    driver_session_id?: number;
    driver_id?: number;
    driver_username?: string;
    vehicle_id?: number;
    route_id?: number;
}

// === Announcement Types (FUTURE – backend needed) ===

interface AnnouncementPayload {
    vehicle_id: number;
    announcement_type: "next_stop" | "current_stop" | "general";
    message: string;
    stop_name?: string;
}

```

6 . Component Architecture

6 . 1 Screen Components

Component	Screen	Purpose
PairingForm	Pairing	

Component	Screen	Purpose
		Code + password input → POST /pair/verify
BusUnlockForm	Bus Device Login	Password input → POST /auth/bus- dashboard/login
DriverLoginForm	Driver Login	Username + password → POST /auth/driver- login
PreRideView	Pre-Ride	Route info + stops + Start button
ActiveRideView	Active Ride	Map + stats + ETA + crowd + progress
RideMap	Active Ride	Leaflet map + live bus marker + stops
StatCards	Active Ride	Speed, crowd, ETA, progress cards
RouteProgressBar	Active Ride	Visual stop-by-stop progress
ETACountdown	Active Ride	Live seconds countdown
CrowdDensityWidget	Active Ride	Level bar + label
AnnouncementPanel	Active Ride	Quick phrases + send form
PostRideSummary	Post-Ride	Trip summary + stats
TripHistoryTable	Post-Ride	Per-stop arrival table
LogoutButton	All	POST /auth/driver- logout + clear

6 . 2 Shared Components

Component	Purpose
ConnectionStatus	WS connected/connecting/disconnected indicator

Component	Purpose
GPSSatelliteIndicator	GPS fix quality indicator
LargeButton	Extra-large touch-friendly button (4 8 px+ height)
LoadingOverlay	Full-screen loading with spinner
ErrorBanner	Top error strip with message
ToastStack	Slide-up notification toasts
CountdownTimer	Reusable live countdown (for ETA)
ConfirmationModal	OK/Cancel dialog (for End Ride)
OfflineBanner	Shows when WS disconnected

7 . Backend Endpoints — Implementation Status

✓ Implemented: Driver Start/End Own Rides

Created `backend/app/api/v1/driver_assignments.py` with `RequireDriver` auth. The driver's identity and vehicle come from their active `DriverBusSession`.

Endpoint	Auth	Body	Purpose
GET <code>/api/v1/driver/assignments/current</code>	JWT (driver)	—	"What ride am I on right now?"
POST <code>/api/v1/driver/assignments/start</code>	JWT (driver)	<code>{ route_id }</code>	Driver starts own ride
POST <code>/api/v1/driver/</code>	JWT (driver)	<code>{ assignment_id }</code>	Driver ends own ride

Endpoint	Auth	Body	Purpose
assignments/ end			

Security: All three endpoints validate that the JWT user's active `DriverBusSession` is linked to the vehicle. A driver cannot end another driver's ride.

✓ Implemented: Trip History Read Endpoints

Created `backend/app/api/v1/trip_history.py`. The `trip_history` table is populated by the telemetry pipeline; these endpoints expose the data.

Endpoint	Auth	Purpose
GET /api/v1/admin/ trip-history/ vehicle/ {vehicle_id}	JWT	Trip history for all assignments of a vehicle (paginated)
GET /api/v1/admin/ trip-history/ assignment/ {assignment_id}	JWT	Trip history for a specific assignment

✗ Missing: Announcements

Nice to Have: Announcements

There's no backend endpoint at all. The existing `bus-dashboard-app/api.ts` has the frontend call defined but it targets nothing.

Fix needed: Announcement model + endpoint

New model: `RideAnnouncement` - id, vehicle_id, announcement_type, message, stop_name?, created_at

New endpoint:

```
### POST /api/v1/announcements
- Auth: JWT (driver role)
- Body: { vehicle_id, announcement_type, message, stop_name? }
- Response: { id, vehicle_id, announcement_type, message,
```

```
stop_name, created_at }
```

- Purpose: Bus driver sends passenger announcement

Future extension: WebSocket broadcast so passengers' mobile apps receive the announcement.

8 . Implementation Plan

Phase 1 : Auth + Shell (Week 1)

Task	Endpoint	Status
Setup Next.js project (new, standalone)	—	—
Pairing screen — code + password	POST /pair/verify	✓
Bus device unlock screen	POST /auth/bus-dashboard/login	✓
Driver login screen	POST /auth/driver-login	✓
Session persistence in localStorage	—	—
Auth state machine (pairing → unlock → driver → pre-ride)	—	—
Logout flow	POST /auth/driver-logout	✓
Root layout + navigation	—	—

Phase 2 : Pre-Ride Screen (Week 1 – 2)

Task	Endpoint	Status
Fetch vehicle data	GET /vehicles/{id}	✓
Fetch route + stops	GET /routes/{route_id}	✓

Task	Endpoint	Status
Display route info panel	—	—
Display stops sequence	—	—
GPS position fetch	GET /vehicles/ positions/{id}	✓
Start Ride button	POST /driver/ assignments/start	✓
Connection status indicator	—	—

Phase 3 : Active Ride — Core (Week 2 – 3)

Task	Endpoint	Status
WebSocket connect + subscribe	WS /ws/mobile	✓
Live map with bus marker	WS vehicle_position	✓
Route stop markers + polyline	cached route data	✓
Speed display	WS speed	✓
ETA countdown (live, with elapsed- time adjustment)	WS eta_payloads	✓
Route progress bar (✓●○)	computed from GPS	✓
Crowd density widget	WS cv_result	✓
Occupancy level (SIM 7 6 0 0 fallback)	WS occupancy_level	✓
GPSSatelliteIndicator	WS lat/lon presence	✓
	WS hook state	✓

Task	Endpoint	Status
Offline detection + banner		
Auto-follow map position	—	—

Phase 4 : End Ride + Summary (Week 3)

Task	Endpoint	Status
End Ride button	POST /driver/assignments/end	✓
Post-ride summary (speed, stops, time)	client-computed	✓
Trip history table	GET /admin/trip-history/vehicle/{id}	✓
Start New Ride	loops back to Pre-Ride	✓

Phase 5 : Announcements + Polish (Week 4)

Task	Endpoint	Status
Announcement panel UI + send	POST /announcements	✗ New backend needed
Quick phrase buttons	—	—
Loading overlays	—	—
Error handling	—	—
Confirmation modals	—	—
Touch-friendly sizing (glove-compatible)	—	—
Responsive mobile/tablet layout	—	—

Phase 6 : Backend Gap Closure (Parallel with Phases 4 – 5)

Driver assignment and trip history endpoints are now implemented (see § 7).

Remaining backend work:

New Endpoint	Complexity	Notes
POST /announcements	Medium	New model + schema + CRUD + route

9 . File Structure

```
bus-dashboard-standalone/  
├── src/  
│   ├── app/  
│   │   ├── layout.tsx                # Root layout  
│   │   ├── (providers + shell)  
│   │   ├── page.tsx                 # Root: redirect  
│   │   ├── based on auth state  
│   │   │   ├── pairing/  
│   │   │   │   └── page.tsx          # Pairing screen  
│   │   │   ├── (first-time setup)  
│   │   │   ├── unlock/  
│   │   │   │   └── page.tsx          # Bus device  
│   │   ├── unlock  
│   │   │   ├── login/  
│   │   │   │   └── page.tsx          # Driver login  
│   │   │   ├── pre-ride/  
│   │   │   │   └── page.tsx          # Pre-ride  
│   │   ├── dashboard  
│   │   │   ├── ride/  
│   │   │   │   └── page.tsx          # Active ride  
│   │   ├── screen  
│   │   │   └── RideMap.tsx           # Leaflet live  
│   │   ├── map component  
│   │   │   └── StatCards.tsx         # Speed/crowd/  
│   │   ├── ETA/progress cards  
│   │   │   └── RouteProgressBar.tsx  # Stop-by-stop  
│   │   └── visual progress
```

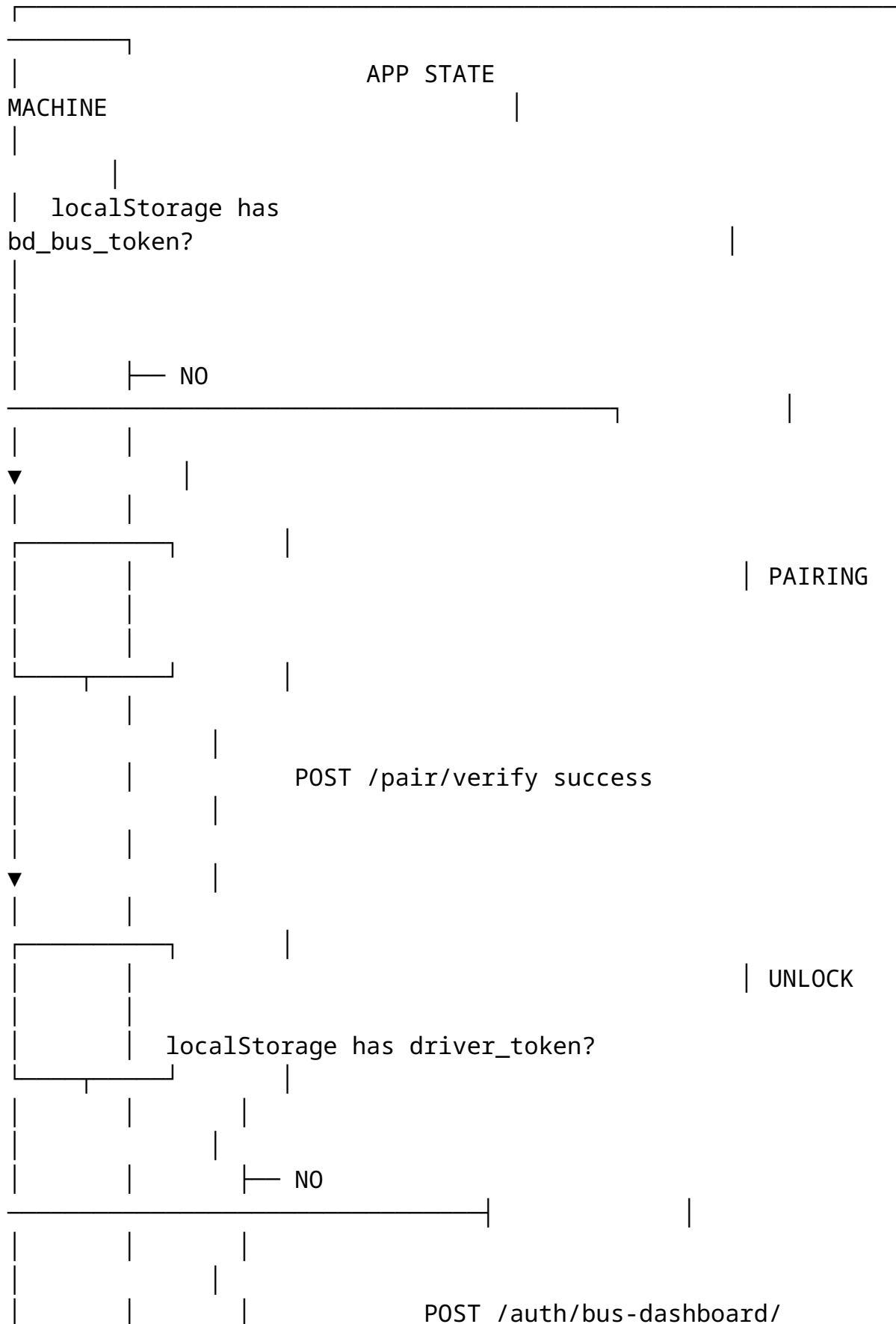
timer	ETACountdown.tsx	# Live countdown
visualization	CrowdDensityWidget.tsx	# Occupancy
announcements	AnnouncementPanel.tsx	# Send
	post-ride/	
summary	page.tsx	# Post-ride
history table	TripHistoryTable.tsx	# Per-stop
	components/	
	ui/	# shadcn/ui
primitives		
	button.tsx	
	input.tsx	
	card.tsx	
	skeleton.tsx	
	...	
	layout/	
	app-header.tsx	# Top bar: bus
ID + status + logout		
	connection-badge.tsx	# WS connection
indicator		
	shared/	
	large-button.tsx	# 48px touch-
friendly button		
	loading-overlay.tsx	
	error-banner.tsx	
	confirmation-modal.tsx	
	countdown-timer.tsx	
	offline-banner.tsx	
	toast-stack.tsx	
	hooks/	
	use-auth.tsx	# Auth state
machine + context		
	use-bus-websocket.ts	# WS connect +
reconnect + subscribe		
	use-live-position.ts	# Derived
current position state		
	use-route-progress.ts	# Computes stop
progress bar state		
	use-eta-countdown.ts	# Live countdown

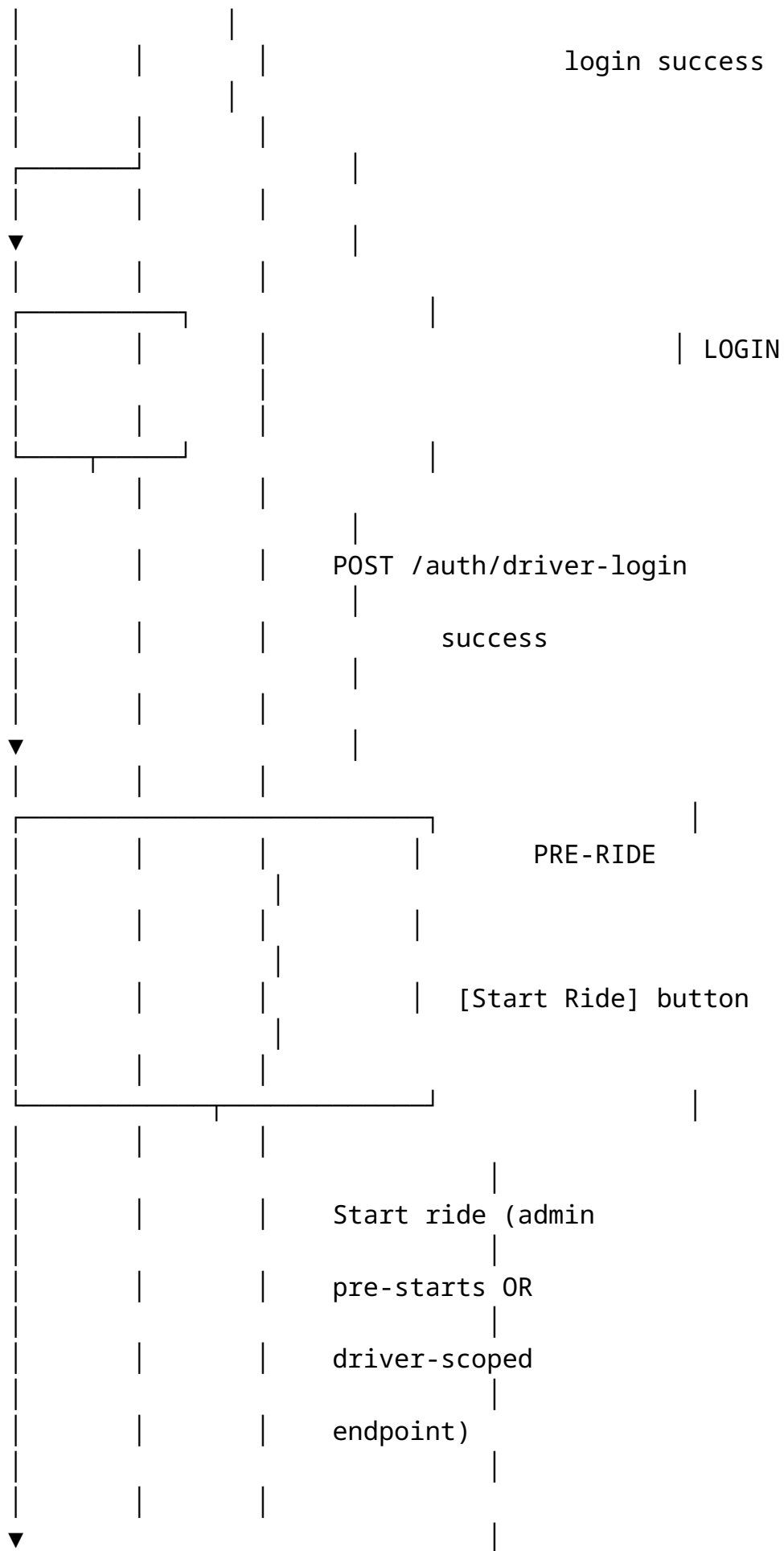
```

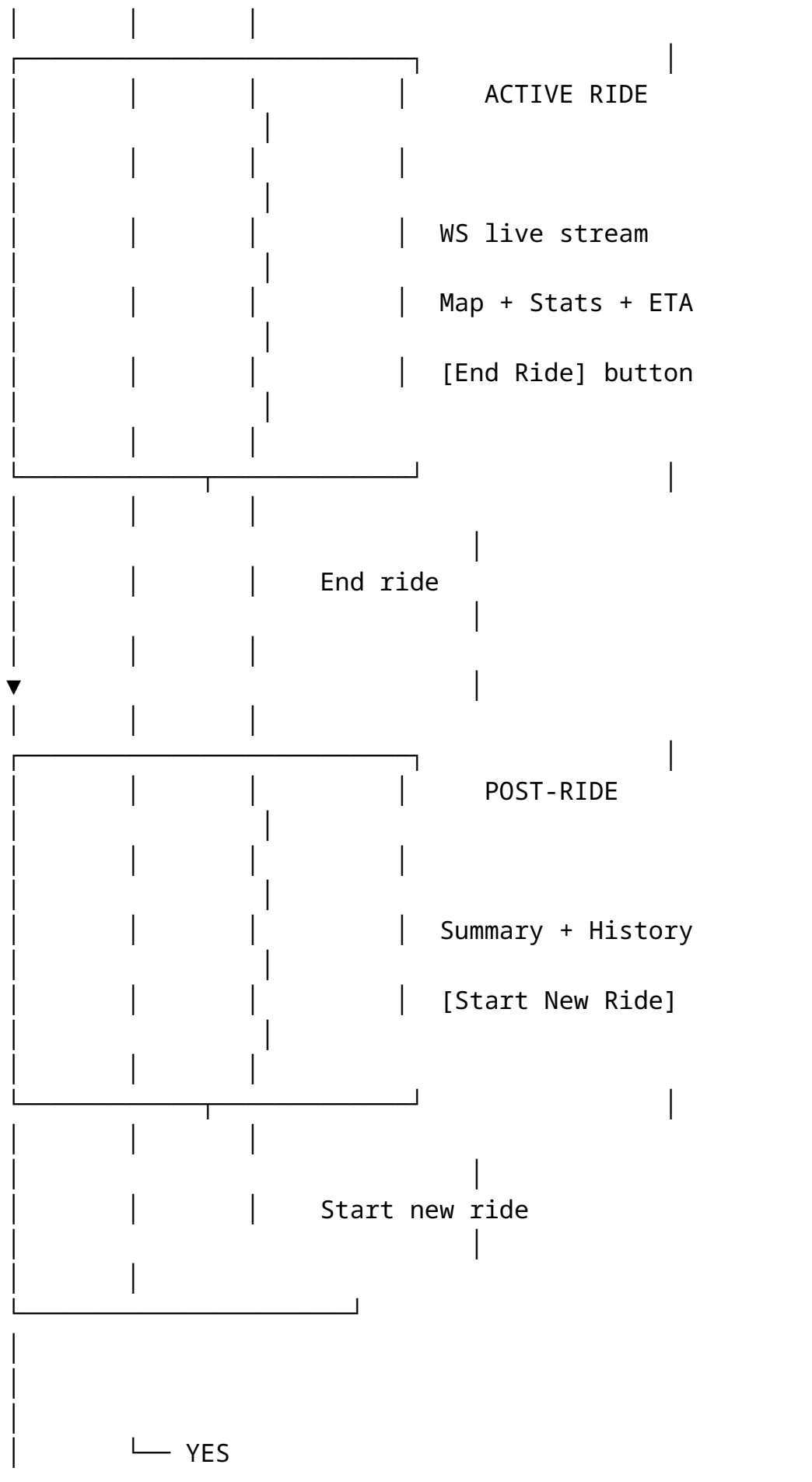
with elapsed-time adjust
|   |   |— use-crowd-level.ts           # Aggregates CV
readings
|   |   |— use-gps-tracking.ts         # Accumulates
distance + samples
|   |   |— use-session-storage.ts      # localStorage
read/write helpers
|   |— lib/
|   |   |— api.ts                      # Axios instance
+ all drivers API functions
|   |   |— auth.ts                    # Token
management + driver session helpers
|   |   |— ws-url.ts                 # WebSocket URL
builder
|   |   |— haversine.ts              # GPS distance
calculation
|   |   |— nearest-stop.ts           # Find nearest
stop index
|   |   |— utils.ts                  # cn() for
classnames
|   |— types/
|   |   |— index.ts                  # All TypeScript
interfaces (§5)
|   |— providers/
|   |   |— auth-provider.tsx         # AuthContext +
useAuth hook
|   |   |— query-provider.tsx       # QueryClient
wrapper
|   |   |— websocket-provider.tsx    # Shared WS
connection
|   |— middleware.ts                 # Route guard:
pairing/ login public; ride private
|   |— .env.local                    #
NEXT_PUBLIC_API_URL=http://localhost:8000
|   |— tailwind.config.ts
|   |— tsconfig.json
|   |— package.json

```

1 0 . State Machine – Complete







#	Method	Path	Auth	Screen	Status
		/vehicles/ /{vehicle_id}			
6	GET	/vehicles/ positions/ /{vehicle_id}	None	Pre-Ride	✓
7	GET	/routes/ {route_id}	None	Pre-Ride	✓
8	GET	/routes/ {route_number}/ etas	None	Active Ride	✓
9	WS	/ws/ mobile? token=	JWT	Active Ride	✓
10	POST	/vehicles/ telemetry	None	Active Ride (GPS send)	✓
11	GET	/health	None	All (heartbeat)	✓

1 1.2 New Endpoints – Status

✓ Implemented

#	Method	Path	Auth	Screen
1 2	GET	/driver/ assignments /current	JWT (driver)	Pre-Ride / Active Ride
1 3	POST	/driver/ assignments /start	JWT (driver)	Pre-Ride
1 4	POST	/driver/ assignments /end	JWT (driver)	Active Ride
1 5	GET	/admin/ trip- history/ vehicle/ {id}	JWT	Post-Ride
1 6	GET	/admin/ trip- history/ assignment/ {id}	JWT	Post-Ride

✗ Still Missing

#	Method	Path	Auth	Screen	Priority
1 7	POST	/ announcements	JWT (driver)	Active Ride	MEDIUM

1 1 . 3 Endpoint Details — New Backend Work

GET /api/v 1 /driver/assignments/current

```
# Returns the current driver's active assignment
# Auth: JWT (driver or admin)
# Logic:
#   1. Get current user from JWT
#   2. Look up active DriverBusSession for this user
#   3. Get vehicle_id from session
#   4. Look up active Assignment for this vehicle
#   5. Return AssignmentOut
# Response: AssignmentOut
```

POST /api/v 1 /driver/assignments/start

```
# Driver starts their own ride
# Auth: JWT (driver or admin)
# Body: { route_id: int }
# Logic:
#   1. Get current user from JWT
#   2. Look up active DriverBusSession → get vehicle_id
#   3. Check no active assignment for this vehicle (409 if exists)
#   4. Validate route_id exists
#   5. Create Assignment(driver_id, vehicle_id, route_id)
# Response: AssignmentOut
```

POST /api/v 1 /driver/assignments/end

```
# Driver ends their own ride
# Auth: JWT (driver or admin)
# Body: { assignment_id: int }
# Logic:
#   1. Get current user from JWT
#   2. Look up active DriverBusSession → get vehicle_id
#   3. Look up assignment by ID
#   4. Verify assignment.vehicle_id matches session.vehicle_id
#   5. End assignment (set end_time, status)
# Response: { status: "ended", assignment_id }
```

GET /api/v 1 /admin/trip-history/vehicle/{vehicle_id}

```
# Read trip history for a vehicle
# Auth: JWT (any authenticated user)
# Query: limit=50, offset=0
# Logic:
#   SELECT * FROM trip_history
#   JOIN stops ON trip_history.stop_id = stops.id
#   WHERE trip_history.assignment_id IN (
#     SELECT id FROM assignments WHERE vehicle_id
#       = :vehicle_id
#   )
#   ORDER BY trip_history.arrival_time DESC
#   LIMIT :limit OFFSET :offset
# Response: list[TripHistoryOut]
```

GET /api/v 1 /admin/trip-history/assignment/{assignment_id}

```
# Read trip history for a specific assignment
# Auth: JWT
# Logic: same as above but WHERE assignment_id
#       = :assignment_id
# Response: list[TripHistoryOut]
```

POST /api/v 1 /announcements

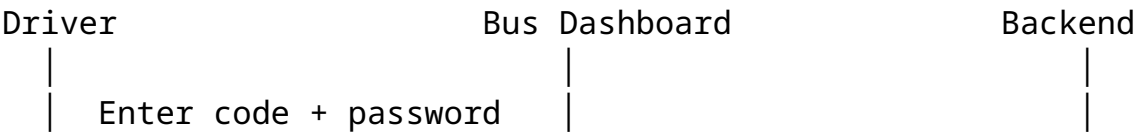
```
# Send passenger announcement
# Auth: JWT (driver or admin)
# Body: { vehicle_id: int, announcement_type:
#         "next_stop"|"current_stop"|"general", message: str,
#         stop_name?: str }
# Logic:
#   1. Validate vehicle exists
#   2. Create RideAnnouncement record
#   3. (Future) Broadcast via WebSocket to subscribed
#       passengers
# Response: { id, vehicle_id, announcement_type, message,
#             stop_name, created_at }
```

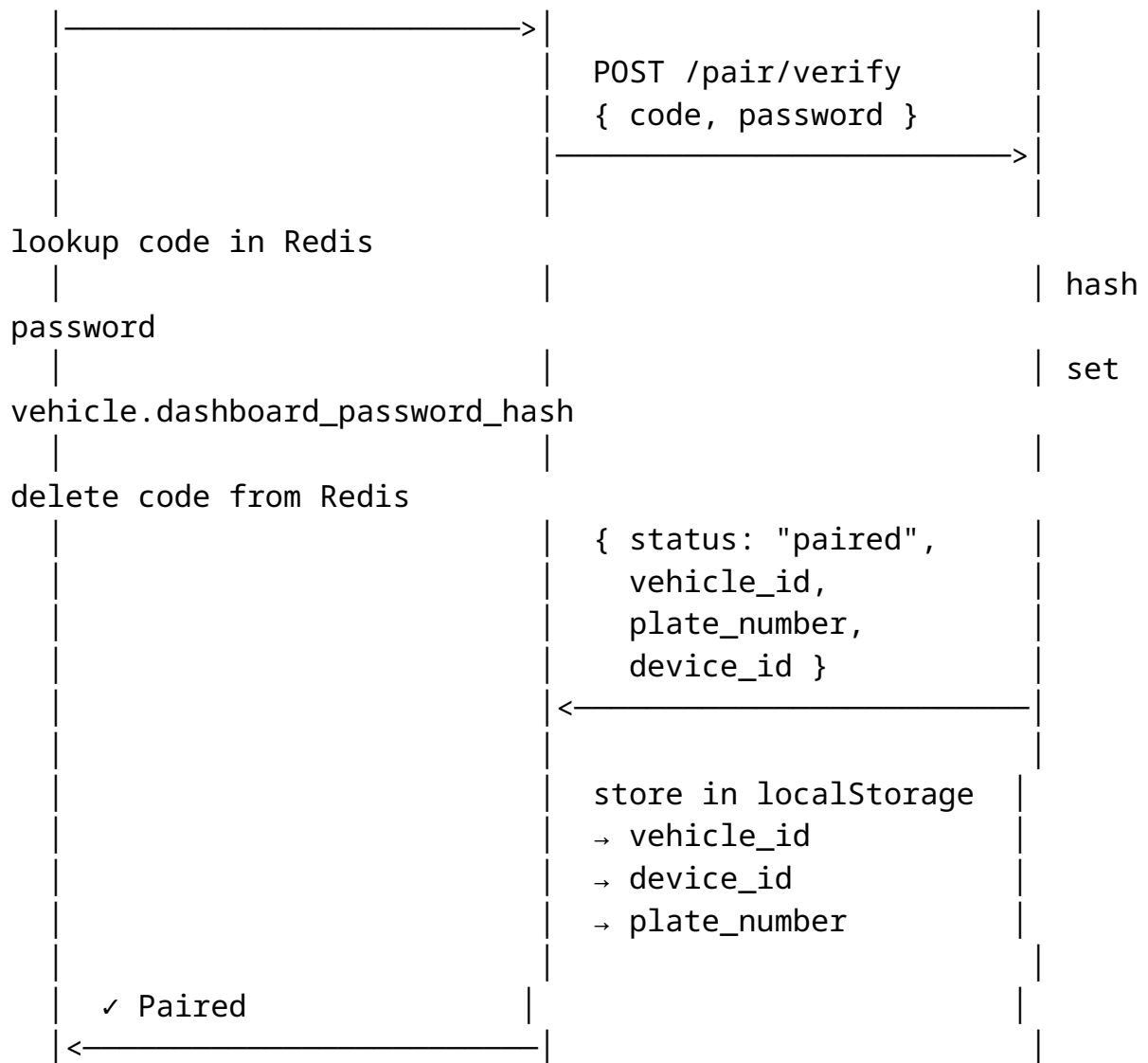
1 2 . Comparison: Bus Dashboard vs. Admin Dashboard

Aspect	Bus Dashboard (this doc)	Admin Dashboard (previous doc)
Users	Bus drivers	System administrators
Purpose	Run a single shift: login → ride → end → logout	Manage entire fleet, analytics, ML
Auth	Device login + driver login (2 - step)	Username/password (1 -step)
Scope	Own bus only	All buses, all data
Screens	7 (pairing, unlock, login, pre-ride, active ride, post-ride, logout)	1 4 (dashboard, map, fleet, routes, trips, users, analytics, crowd, ML, settings)
API calls	1 1 existing + 6 new	6 7 (all admin endpoints)
WebSoc ket	/ws/mobile (filtered by route)	/ws/live (all fleet)
Data entry	Announcements, start/end ride	Vehicle/route/user CRUD, ML training
Analytic s	None	6 chart types
Complex ity	Low — single-user, single-bus	High — multi-user, multi-bus

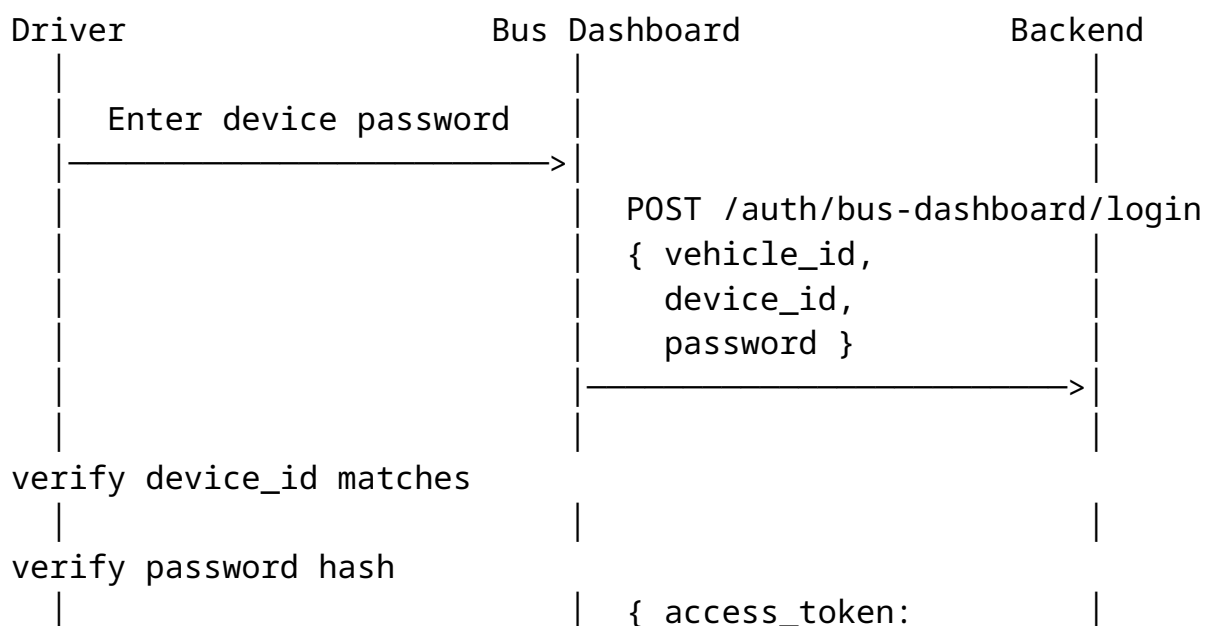
1 3 . Appendix — API Call Sequence Diagrams

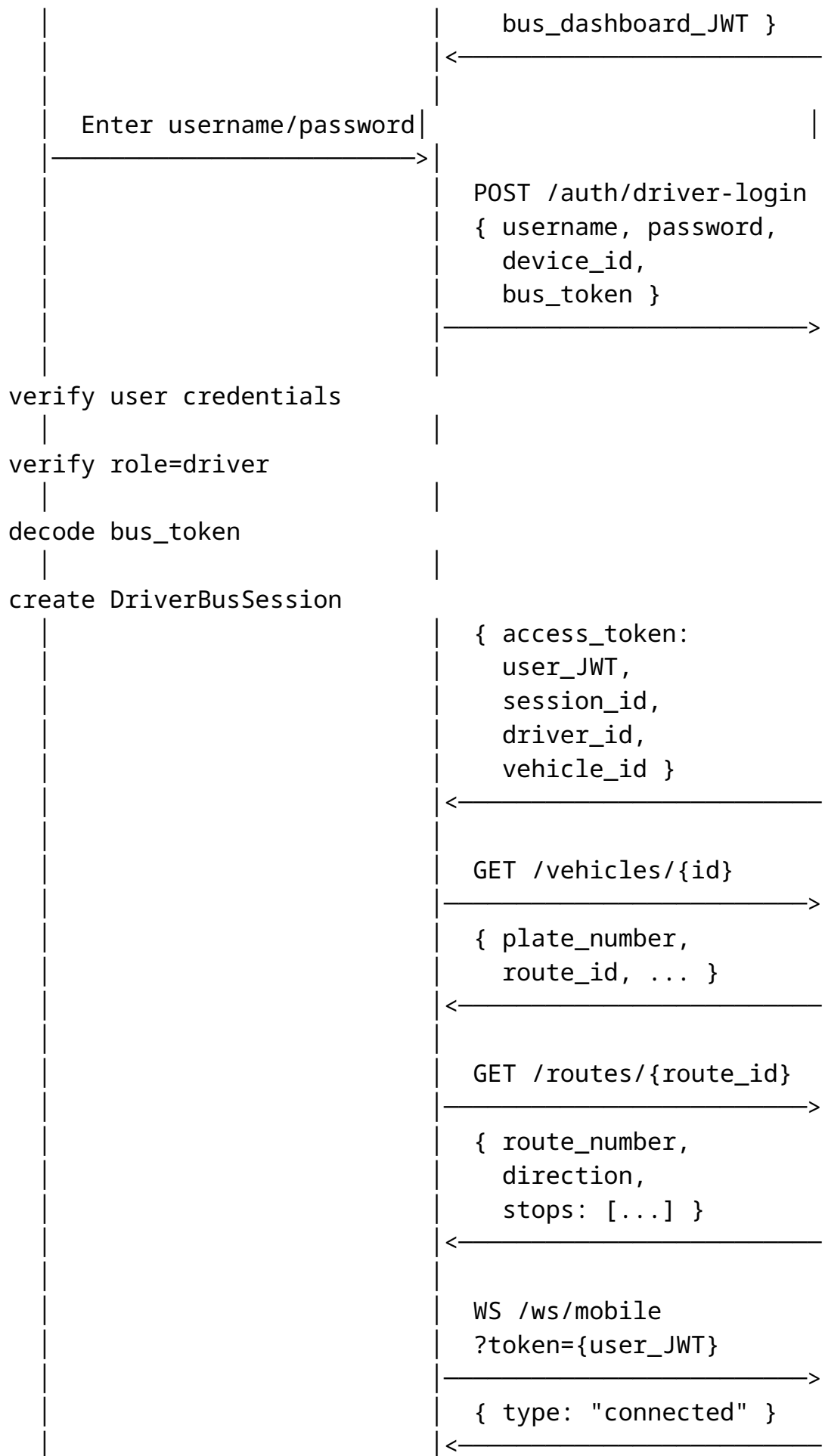
1 3 . 1 First-Time Setup (Pairing)

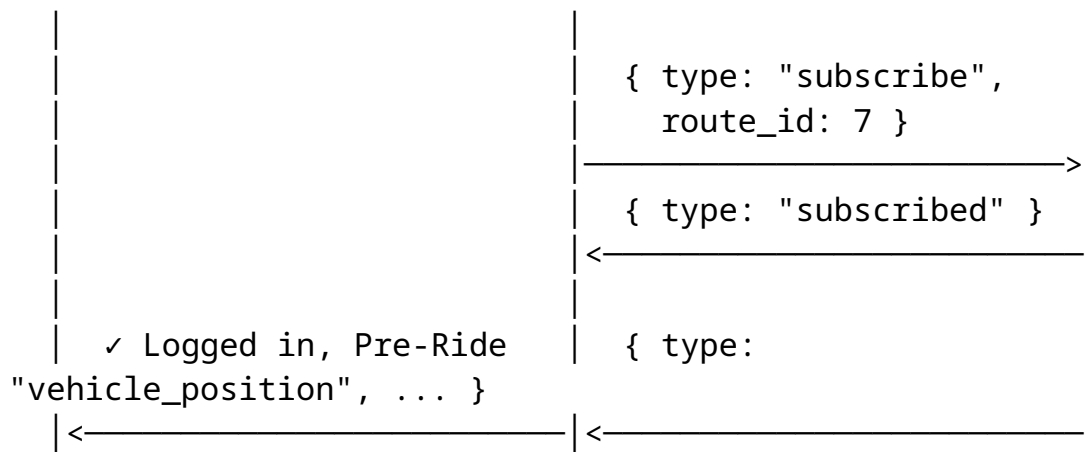




1 3 . 2 Normal Login Flow







1 3 . 3 Active Ride Loop

